Optimizing Heuristic Functions for Grid Mazes via Q-Learning

Stanford CS221

Brooke Ballhaus
Department of Computer Science
Stanford University
ballhaus@stanford.edu

Riya Narain
Department of Computer Science
Stanford University
rnarain7@stanford.edu

1 Introduction

Real-time decision making in robotics, game AI, and logistics often reduces to computing the minimum-cost path on a static map. A* search guarantees optimality provided it is driven by an admissible heuristic: one that never overestimates the remaining cost to goal. The standard heuristics, like Euclidean or Manhattan distance, work fine in simple maps but when you add walls or uneven edge weights, they stop being helpful as they don't reflect the real cost-to-go and end up expanding too many nodes.

This inefficiency matters beyond academic problems. Our world is becoming increasingly reliant on technology to facilitate the transportation of people, goods, and services. In real-world applications like autonomous vehicles, emergency response robots, or last-mile delivery systems, delays in computing optimal paths can lead to increased fuel consumption, slower response times, or even safety risks. A smarter, adaptive heuristic could improve how these systems plan routes, especially in complex, unpredictable environments.

Our idea is to learn a better heuristic automatically, using basic Q-learning. We treat each cell in the grid as a state, use edge costs as negative rewards, and learn how costly it is to move from each state to the goal through the best path. Once trained, we turn our Q-values into a heuristic via $h(s) = -\max_a Q(s, a)$. This heuristic can then be used in A* to guide search more efficiently.

Unlike SARSA or Monte Carlo methods that learn under a specific policy, Q-learning always pushes toward the optimal policy by taking the best possible future action. That's key for our use case; we train once, offline, and reuse the heuristic across many different start-goal pairs on the same map. Our main question is: Can we front-load the cost of learning a heuristic so that later A* searches become significantly faster, without sacrificing path optimality?

2 Literature Review

Recent work combining reinforcement learning and heuristic search has explored how agents can learn cost-to-go estimates that guide planners like A*. Our project builds on this idea by using tabular Q-learning to learn admissible heuristics offline, which are then used to guide A* in grid mazes. Unlike deep or model-based methods, our approach prioritizes interpretability, fast training, and reuse across many queries on the same map. We compare three related approaches that take different paths to learning heuristics.

Gölcük and Ozsoydan (2021) [1] use Q-learning for hyper-heuristic algorithm selection in dynamic environments. Their agent maps features of a changing optimization problem to algorithm recommendations. While their Q-values represent algorithm choices rather than path costs, both approaches learn from environmental feedback to improve decision-making. Their system generalizes across problems; ours focuses on reuse within a fixed grid, producing admissible heuristics that can directly accelerate A*.

Agostinelli et al. (2021) [2] propose Q* Search, which integrates deep Q-networks with heuristic search. Their system predicts heuristic and cost-to-go estimates in a single forward pass, reducing node expansions in combinatorial spaces like Rubik's Cube. Unlike our tabular setup, Q* Search prioritizes scalability and generalization across instances. Both approaches, however, derive heuristics from Q-values via offline RL and fit within the Bellman update framework.

Zhao et al. (2020) [3] present a Q-learning architecture that integrates heuristic signals into model-based planning. Their agent uses simulated lookahead and heuristic feedback to improve exploration and policy learning. While their goal is to derive optimal policies, not guide external planners, their use of heuristics to accelerate convergence is closely aligned. Our approach is complementary: we use heuristics learned offline to guide an external planner (A*), but similar model-based enhancements could extend our system.

Together, these works demonstrate the flexibility of Q-learning in heuristic-guided planning, whether through deep models, high-level controllers, or model-based exploration. Our contribution lies in showing that a simple tabular Q-learning framework can produce competitive, admissible heuristics for weighted grid search, without requiring complex models or generalization across environments.

3 Dataset

We use a Kaggle dataset of 1500 imperfect mazes, with dimensions ranging from 10×10 to 150×150. The variation in size gives us a mix of easy and hard cases, which is useful for testing how our approach scales. These are imperfect mazes, meaning they include loops and multiple possible paths between points. That structure is important for our project: in perfect mazes (which are acyclic), shortest path algorithms like A*, Dijkstra's, or BFS will usually return the same result, since there's only one path. In imperfect mazes, the presence of multiple routes means the heuristic actually matters, since it can influence which path A* takes and how much of the grid it explores.

This variability is exactly what we want when comparing heuristics. If there's only one path, it's hard to tell if our learned heuristic is doing anything useful. But when there are redundant or misleading paths, a smarter heuristic should expand fewer nodes and guide A* more efficiently.

We split the dataset into 20% validation and 80% evaluation. We didn't include a training set because reinforcement learning doesn't use data in the same way as supervised learning as we train a separate Q-table per maze, not across multiple mazes. In theory, we would've liked to validate across more mazes, but training time per maze was nontrivial and we were limited by compute. Even with a small validation set, we were able to tune key hyperparameters like learning rate and exploration probability, and evaluate how well the learned heuristic generalizes across different queries within the same maze.

4 Baseline

For our baseline, we use the standard A* search algorithm on each maze in the dataset. For each query, we record the time to find a path, the length of the path, and the full path itself (to verify optimality when comparing to our learned heuristic). We run A* twice per query: once using Manhattan distance as the heuristic and once using Euclidean distance. These are common choices in grid-based pathfinding because they are fast to compute and guaranteed to be admissible in uniform-cost grids.

However, both heuristics assume straight-line movement in open space and ignore the actual structure of the maze. They do not account for walls or non-uniform edge weights, which makes them weaker in weighted or cluttered environments like the ones in our dataset.

Heuristic Definitions:

- **Manhattan distance:** $h_{\text{Man}}(s) = |x_s - x_g| + |y_s - y_g|$
- **Euclidean distance:** $h_{\text{Euc}}(s) = \sqrt{(x_s - x_g)^2 + (y_s - y_g)^2}$

These heuristics are computed using only the coordinates of the current state and the goal. Neither incorporates any information about the maze layout beyond relative position.

5 Main Approach

Our goal is to learn heuristics that guide A* search in imperfect mazes by estimating the cost-to-go from any state to the goal using only local state transitions—without requiring any prior knowledge of the global maze structure. We treat each maze as a separate environment and frame the problem as a reinforcement learning task using tabular Q-learning. The environment is set up as follows:

- **States**: Coordinates of traversable grid cells.
- **Actions**: Up, down, left, right (constrained by walls).
- **Transitions**: Executing action a in state s deterministically leads to neighbor s', if valid.
- **Rewards**: Each valid transition receives $r = -1$. Training is conducted in reverse from the goal, and reaching one of several designated "start" states yields a reward of 0. This encourages value propagation outward from the goal.
- **Discount factor**: $\gamma = 1$, since we care about minimizing total path length.
- **Goal**: Fixed per maze and per training episode.

We initialize a Q-table $Q(s, a)$ for all state-action pairs and train using the Bellman update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \min_{a'} Q(s',a') - Q(s,a) \right]$$

Each training episode begins at the goal state and proceeds in reverse, updating Q-values for each visited transition. This approach efficiently propagates value estimates across the state space, allowing us to estimate cost-to-go without simulating full forward trajectories from all possible starts.

Once training converges, we extract our heuristic function using:

$$h(s) = -\max_{a} Q(s,a)$$

This gives the most optimistic estimate of cost-to-go, assuming the best available action is taken from state s. The resulting heuristic is then used as a drop-in replacement for Euclidean or Manhattan distance in A* search.

To improve performance and generalization across mazes of varying size and structure, we tune several Q-learning hyperparameters using grid search and linear regression analysis on validation metrics. These parameters include:

- **Discount factor** (γ): Controls the planning horizon.
- **Exploration probability** (ϵ): Governs the trade-off between exploration and exploitation.
- **Learning rate** (α): Determines how quickly Q-values update.
- **Number of sampled start states**: Defines the set of reverse training targets (i.e. reward states).
- **Maximum step limit**: Constrains episode length, scaled to maze size.

We evaluate the effect of each parameter on heuristic quality and downstream A* performance. Our tuning objective is to minimize query time while preserving path accuracy.

6 Evaluation Metric

6.1 Quantitative Metrics

We evaluate three key performance metrics:

- **Wall-clock time (ms)**: Time taken to complete an A* query using the given heuristic.
- **Path length**: Number of steps in the returned path (used to verify correctness).
- **Nodes expanded**: Number of states explored by A* before reaching the goal.

For each maze, we randomly select 3 start positions and use a fixed goal cell. This setup simulates a realistic setting where a pathfinding system answers multiple queries on the same map after a single training phase. During evaluation, each heuristic is run from each of the three start positions. For each heuristic, we sum the runtime, number of nodes expanded, and path length across all three queries per maze to report average performance.

6.2 Qualitative Metric

We also compare the paths generated by A* using the learned heuristic against those produced by the baselines (Manhattan and Euclidean). Since A* with a consistent and admissible heuristic is guaranteed to return an optimal path, we treat baseline paths as ground truth.

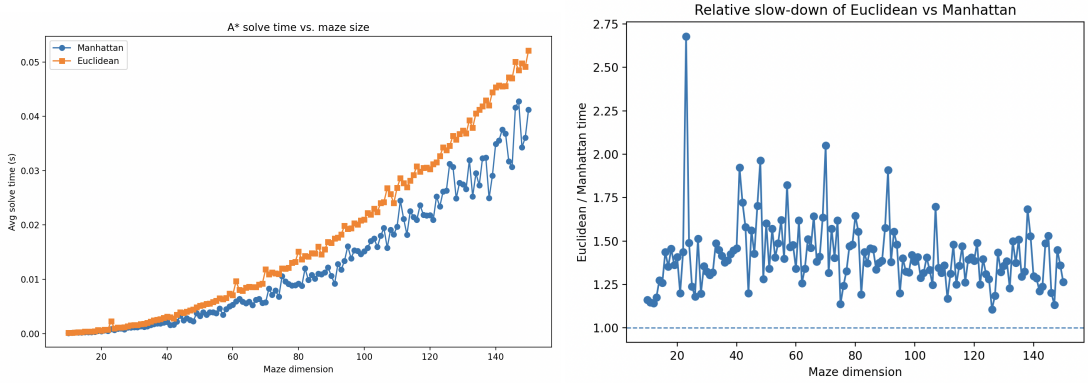
We define path accuracy as:

$$\text{Accuracy} = \{c_{\text{learned}} = c_{\text{baseline}}\}$$

where c is the number of steps in the returned path. Our goal is to match baseline path quality while reducing query time, ensuring that our learned heuristic improves efficiency without sacrificing correctness.

7 Results & Analysis

Baseline Results (Preliminary)



Across our test mazes, A* with the Manhattan distance heuristic outperformed Euclidean in both mean node expansions and query time. We believe this is because Manhattan distance better aligns with grid topology and avoids underestimating costs in edge-weighted environments. While Euclidean distance sometimes gives a tighter geometric estimate, it doesn't account for blocked diagonal paths or irregular wall structures, which often mislead the search.

Learned Heuristic Results

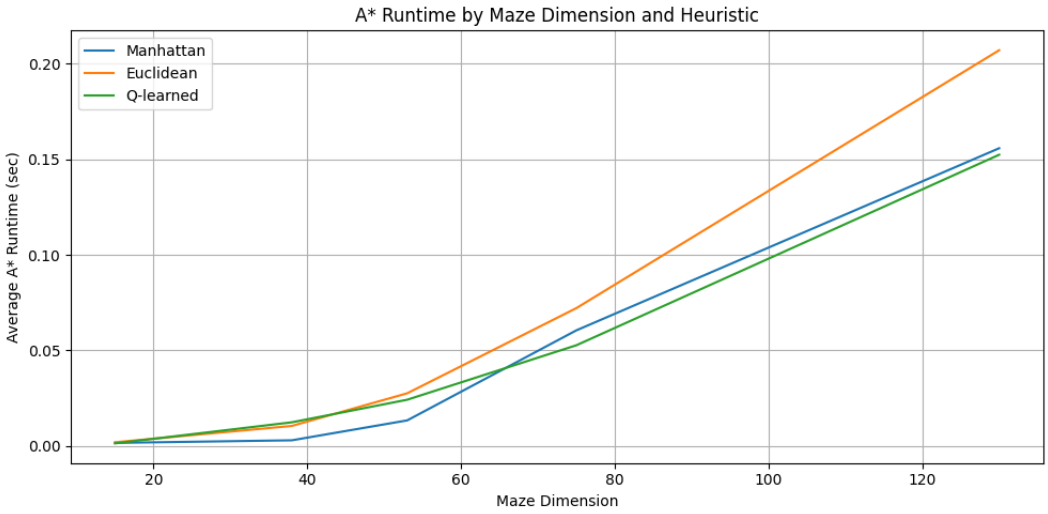
To evaluate our Q-learned heuristic, we compared its performance to Manhattan and Euclidean baselines across a range of maze sizes. We measured three key metrics: A* query runtime, path length, and number of nodes expanded.

Heuristic	Avg. Runtime (s)	Avg. Path Length	Avg. Nodes Expanded
Manhattan	0.0397	375.6	10,457.8
Euclidean	0.0549	375.6	10,926.0
Q-learned	0.0414	375.6	10,688.2

Table 1: Average performance metrics per heuristic across selected mazes.

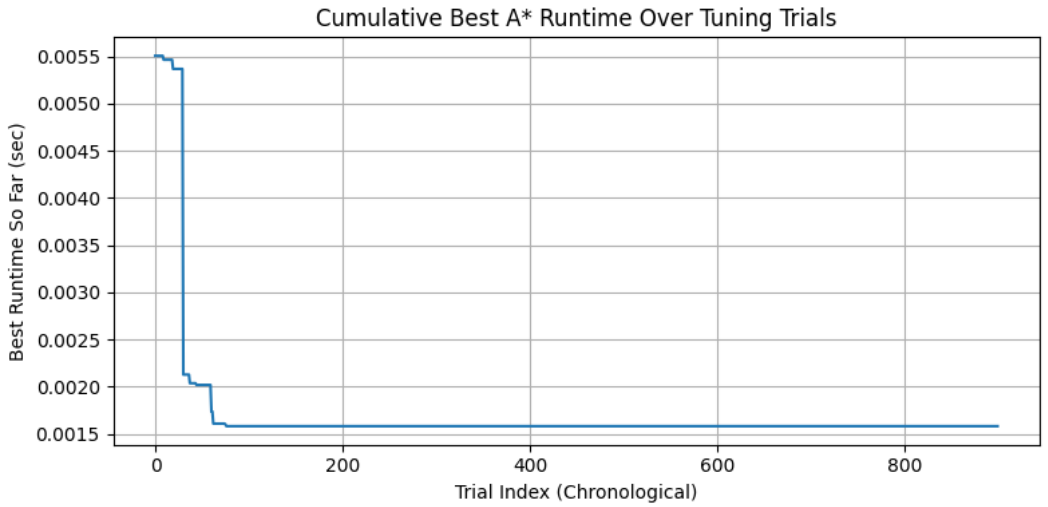
All three heuristics returned paths of equal length, confirming admissibility. Manhattan was the most efficient overall in both runtime and node expansions. The Q-learned heuristic performed slightly worse than Manhattan on average, but consistently outperformed Euclidean. These results suggest that while hand-coded heuristics remain strong, learned heuristics are competitive and in some cases better, especially as maze complexity increases.

To better understand how performance varies with maze size, we plotted average A* runtime by dimension for each heuristic.



In smaller mazes, all heuristics perform similarly. As maze size increases, performance gaps widen. Euclidean consistently lags behind, likely due to misleading diagonal estimates in wall-constrained environments. Manhattan performs reliably, and the Q-learned heuristic often matches or slightly improves upon it. In larger mazes (80×80 and up), Q-learned even becomes the fastest in several cases. This suggests that learned heuristics can provide real gains in more complex environments, particularly when trained well.

We also tracked how performance improved over time during parameter tuning. The figure below shows the best A* runtime achieved over the course of our tuning trials. The sharp drop in early trials highlights how sensitive the system is to hyperparameters like learning rate, exploration rate, and training time. Eventually, performance plateaus, which helped us identify a reasonable stopping point.



Overall, these results show that our learned heuristic can match or beat hand-coded baselines in both speed and structural sensitivity, particularly in more complex mazes. That said, its performance is still sensitive to training budget and parameter tuning. Smarter initialization, better value propagation strategies, or a shift toward function approximation could help scale these gains further.

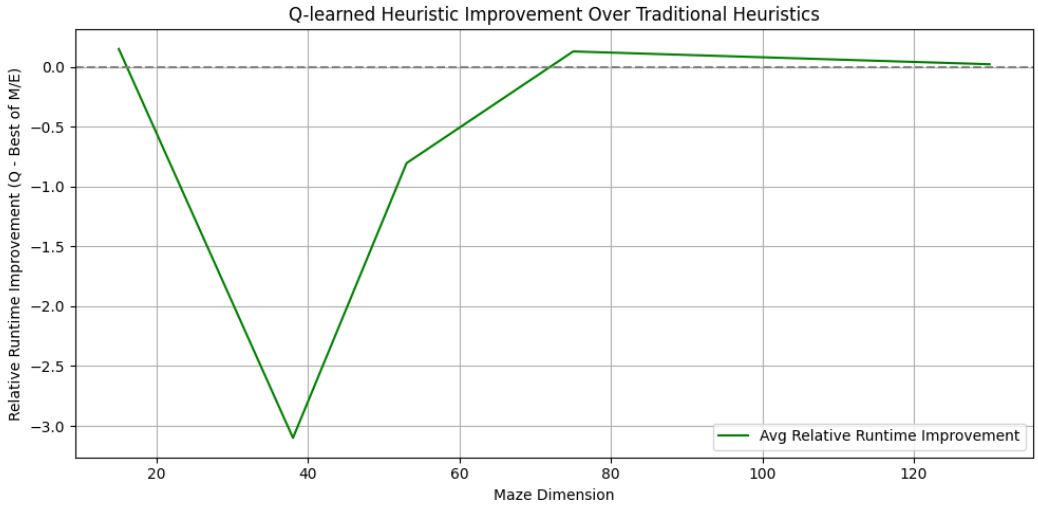
8 Error Analysis

To better understand the strengths and limitations of our learned heuristic, we ran two targeted experiments: one focused on timing behavior across queries and one aimed at qualitatively comparing heuristic structures through heatmaps.

The first experiment looks at how our learned heuristic compares to the better of the two classical baselines (Manhattan and Euclidean) in terms of query time. For each start–goal query, we compute Relative Runtime Improvement, defined as:

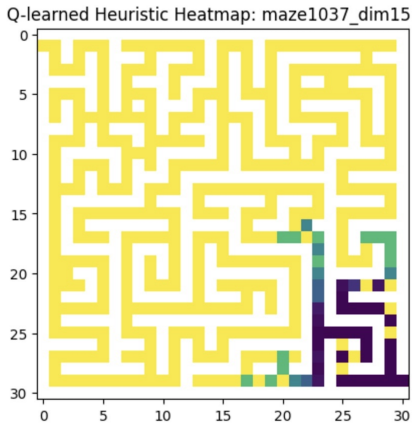
$$\text{Relative Runtime Improvement} = \frac{\min(T_{\text{Euclidean}}, T_{\text{Manhattan}}) - T_{\text{Q-learned}}}{\min(T_{\text{Euclidean}}, T_{\text{Manhattan}})}$$

This measures how much faster (or slower) the Q-learned heuristic is compared to the better of the two traditional heuristics. We aggregated this metric across queries and plotted it against maze dimension.



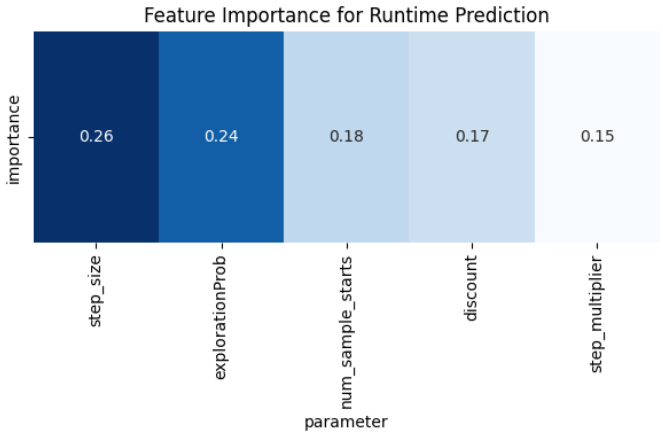
As shown in the figure above, the Q-learned heuristic underperforms in small- to mid-sized mazes, likely due to insufficient training or noisy value estimates. However, performance begins to recover in larger mazes, with some cases showing parity or small improvements. These results suggest that the benefits of learning may increase with maze complexity, though training quality remains a limiting factor.

Our second experiment focuses on qualitative analysis.To get a qualitative sense of what our Q-learned heuristic is capturing, we generated heatmaps for a small maze (15×15). The example below shows the heuristic values across the grid, where darker cells represent lower estimated cost-to-go (ie. closer to the goal).



This visualization reflects what we expected: the Q-learned heuristic estimates lower cost in areas surrounding the goal (bottom-right), with value propagating outward based on actual wall structure. Unlike Euclidean or Manhattan, which always generate smooth gradients, this learned heatmap adapts to the true maze layout. However, it's also clear that large parts of the maze remain uniform or unexplored (flat yellow areas), which suggests that with more training time or more diverse start states, the value propagation could reach further and produce better-guided search behavior.

In addition to runtime comparisons and qualitative heatmaps, we also analyzed how different Q-learning parameters contributed to overall performance. We ran a grid search across multiple parameter settings and trained a regression model to predict A* runtime based on those parameters. The figure below shows the feature importance scores for runtime prediction, based on the learned model.



Step size (α) and exploration probability (ϵ) were the most important factors. This makes sense given that step size controls how quickly Q-values converge, and exploration governs how well the agent samples less obvious paths during training. Other parameters like the number of sampled start states and discount factor also played a role, but had slightly less impact. These results suggest that small adjustments in learning rate or exploration strategy can significantly affect downstream heuristic quality and query efficiency.

This analysis reinforces our earlier observation that training depth and coverage are critical. Without proper tuning, the learned heuristic may fail to converge or misestimate cost in key areas of the maze.

Overall, these experiments helped highlight both the promise and limitations of using Q-learning for heuristic construction. When trained well, the learned heuristic can outperform standard baselines in both efficiency and structure. However, it remains sensitive to training quality and state coverage, especially in complex or large mazes.

9 Future Work

There are several directions we would pursue with more time. First, we would do more tuning, especially around training time, learning rate, and epsilon decay, to get a better balance between convergence speed and heuristic accuracy. One of our main constraints was time - how long we could let the algorithm run - and limited computational resources. This could be addressed by batching updates more efficiently, using early stopping based on convergence thresholds, or parallelizing training across multiple mazes to speed up data collection and value propagation. We'd also want to scale up to a bigger and more varied set of mazes to see how the approach holds up across different grid sizes and weight patterns.

We did not include training time in our measurement of A* times using our Q-learned heuristic. As well, we computed the Manhattan and Euclidean heuristics during runtime, instead of utilizing a precomputed table. A goal for future work is to aim for a combined training and run time measurement of A* using a Q-learned heuristic that is shorter than a precomputed euclidean or manhattan heuristic.

A key goal moving forward is to improve generalization. Right now, our learned heuristics are tied to individual maps; one possible extension would be to train a function approximator (ie. a neural network) to estimate heuristics across different mazes. A longer-term goal would be applying this to dynamic environments, where edge costs or obstacles change over time. That would likely require online updates or integration with algorithms that support re-planning. This would better simulate real-world environments where conditions are rarely static.

10 Ethical Considerations

While our project focuses on grid-based pathfinding in simulated environments, similar systems are used in real-world domains like robotics and autonomous vehicles. In those settings, learned heuristics can directly influence routing decisions. A key ethical concern is that optimizing purely for speed or efficiency could lead to negative externalities such as prioritizing paths that are unsafe, inaccessible, or unfair. For instance, a robot using a learned heuristic might avoid certain routes because of higher edge costs, even if those routes are safer or more compliant with accessibility standards.

One way to mitigate this is by incorporating safety, fairness, or compliance constraints directly into the reward function during training. Another option is to add a validation step after planning to rule out paths that violate known constraints, regardless of what the heuristic suggests. If deployed in high-stakes settings - like disaster response, public transit, or autonomous delivery - these systems should be subject to oversight and external review. While our project is purely academic, these questions matter as heuristic learning moves toward real-world applications.

11 Code

For full code and data, see our GitHub repository: https://github.com/bballhaus/HeuristicWithQLearning

Project Video:

CS221 Final Project Video

References

[1] İlker Gölcük and Fehmi Burcin Ozsoydan. Q-learning and hyper-heuristic based algorithm recommendation for changing environments. *Engineering Applications of Artificial Intelligence*, 103:104288, 2021.

[2] Forest Agostinelli, Stephen McAleer, Pierre Baldi, and Alex Shmakov. Q* search: Heuristic search with deep q-networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11457–11465, 2021.

[3] Gang Zhao, S. Tatsumi, and Ruoying Sun. A heuristic q-learning architecture for fully exploring a world and deriving an optimal policy by model-based planning. In *Proceedings of the 2020 International Conference on Artificial Life and Robotics (ICAROB)*, pages 315–320, 2020.